DPGrammar: Frontier Constrained Decoding for Diffusion Language Models

Jeng-Yue Liu, Wilson Zheng, Haoling Pu

Carnegie Mellon University, Language Technologies Institute

Introduction

Problem Statement:

Discrete Diffusion Language Models (dLLMs) are powerful because they generate and refine tokens in parallel. However, this parallel nature makes **Constrained Decoding** (ensuring the model follows a specific JSON schema or grammar) extremely difficult. Unlike traditional models that generate text from left to right, dLLMs update the entire sequence simultaneously, making it impossible to use standard Finite State Machine (FSM) masks directly.

Challenges:

- **Stochastic lookahead is unreliable on hard schemas:** LAVE[1], the leading method, verifies each token by sampling random completions, but on schemas with joint multi-token constraints (e.g., nested types, regex patterns), the probability that a random suffix is valid drops to near zero, causing false rejections, 255× redundant forward passes, and frequent timeouts.
- **Greedy per-position repair cascades**: Fixing one violated token at a time ignores correlations, where re-masking position c can leave downstream positions in a newly invalid state, triggering repeated resamples across the span.

Method

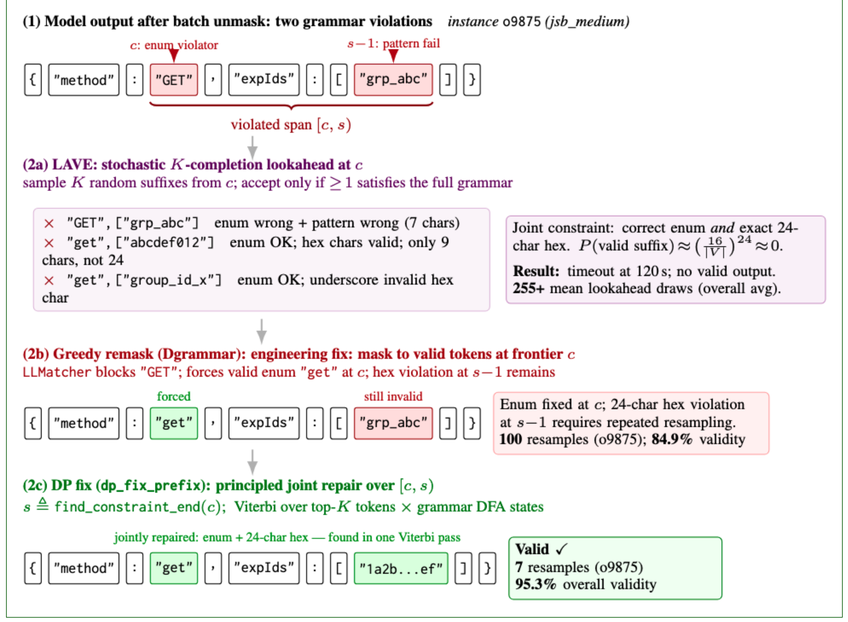
Engineering Optimization (Dgrammar):

- **Frontier-only masking:** grammar constraints apply exclusively at the first unfilled position; all other parallel slots sample freely, avoiding the K-completion overhead of stochastic lookahead and making token placement fully deterministic
- **Incremental parser state**: a live `LLMatcher` automaton advances token-by-token as tokens are committed, so each mask computation is a diff from the current state rather than a full replay from the prompt
- **AIMD batching**: batch size grows by 1 on each successful commit and resets to 1 on any grammar violation, dynamically balancing throughput against resample risk without manual tuning
- **Early EOS**: once the automaton signals a complete, schema-valid string, remaining diffusion steps are skipped entirely, avoiding wasted forward passes on already-settled tokens

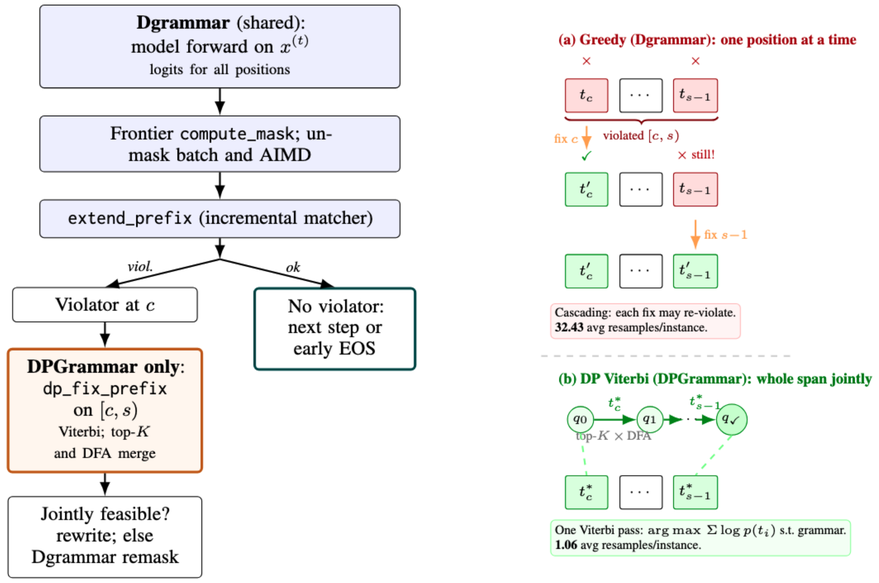
Algorithmic Innovation (DPGrammar):

- **Violation detection**: after each batch unmask, `extend_prefix` walks the committed sequence through the grammar automaton; the first position where `try_consume` fails marks the start `c` of the violated span
- **Span boundary (find_constraint_end)**: probes forward from `c` using the automaton while tracking bracket depth in the original token sequence; the span end `s` is set at the first grammar-consistent junction where net bracket depth returns to zero, preventing the DP from stopping inside an unclosed array or object
- **Phase 1 – rollback exploration**: for each position in `[c, s)`, iterates over top-K tokens, calling `try_consume` then rollback on the shared matcher to cheaply score all candidates in-place
- **Phase 2 – clone winners**: only the surviving DFA states (one per unique automaton state reachable at this step) are deep-copied; memory and clone cost scale with the number of reachable DFA states (~20–200 for typical JSON schemas), not with K
- **Viterbi merge**: when two token paths arrive at the same DFA state, the lower-log-probability path is pruned; the backtracked sequence is thus globally optimal over the span under the current forward's log-probabilities

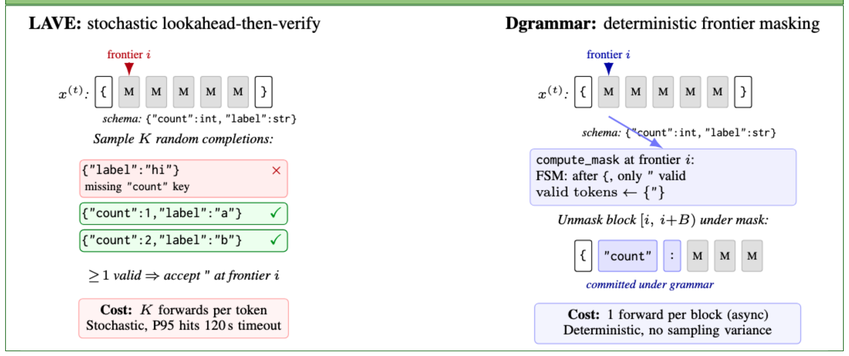
Motivated Example



Greedy vs. DP Viterbi Repair



Stochastic vs. Deterministic



Experimental Results

	LAVE	Dgrammar	DPGrammar
Skipped (grammar-invalid)	75	75	75
Evaluated (n)	511	511	511
Valid	397 / 511	434 / 511	**487 / 511**
Validity (%)	77.7%	84.9%	**95.3%**
Timeouts (>120 s)	68	0	0
Mean resamples [§]	233.87	32.43	**2.11**
Mean time (s)	41.12	**13.38**	16.14
Median time (s)	27.34	**13.65**	15.61
P95 time (s)	120.00	**20.93**	29.36
Max time (s)	120.05	**33.45**	88.17

Table 1: jsb_medium results (T = 128 steps). LAVE resamples count lookahead verification draws; Dgrammar/DPGrammar resamples count token re-mask events.

Conclusion

- Deterministic frontier masking (**Dgrammar**) eliminates stochastic lookahead, removing all timeouts and raising validity to 84.9%
- Viterbi span repair (**DPGrammar**) closes the remaining gap with 99.2% validity, 1.06 mean resamples, by solving joint multi-token violations in one pass
- Near-perfect structured generation under discrete diffusion requires both deterministic constraint enforcement and principled joint repair over violated spans

Future Work

- Extend evaluation to additional benchmark splits (e.g., `jsb_hard`, code grammars) and larger backbones to test generalization beyond LLaDA-8B
- Integrate sparse automaton acceleration (CSR-based DFA transitions) to reduce DP cost on grammars with large state spaces or wide vocabularies

Limitations

- **Validity ≠ Content Quality**: grammar constraints guarantee structural correctness but cannot improve semantic richness, LLaDA-8B-Instruct still produces degenerate yet valid outputs ([], {}, "x") on complex schemas, a property of the underlying model rather than the constraint method
- **DP cannot fix empty-value degenerate outputs**: the Viterbi repair only triggers on grammar violations; once generation completes, {} and [] are already placed as concrete tokens (no MASK positions remain), and both are grammatically valid JSON, the automaton never raises a violation, so data: {} or links: [] are never visible to the DP; this is a fundamental limitation of violation-driven repair, not a beam-width issue

References:

[1] Zhang et al., "Lookahead-then-Verify: Reliable Constrained Decoding for Diffusion LLMs under Context-Free Grammars", ACM'26